

Leading Projects

Data Engineer

김성호

Table of contents

(목차 클릭 시 해당 페이지로 이동합니다.)

01 Data Warehouse

1. 데이터 웨어하우스 설계
2. item-stat Data 모델링

02 Data pipeline

1. Data Compaction
2. Spark – Dynamic Resource Allocation 도입

03 Dashboard Server

1. 대시보드 API 로딩 속도 개선 (계층 별 캐싱)

01. Data Warehouse

데이터 웨어하우스 설계

01. Data Warehouse

데이터 웨어하우스 설계

Problems.

- 소싱한 데이터를 처리하고, 저장하는 파이프라인이 전부 파편화 되어있음
- 다른 테이블에 비슷한 데이터가 들어가기로 하는 등 중복되는 로직이 존재
- 테이블 명 컨벤션 파편화

01. Data Warehouse

데이터 웨어하우스 설계

Solutions.

- Data Lake → Data Warehouse → Data Mart로 논리적 구분
- S3 경로를 통해 물리적으로 구분

01. Data Warehouse

데이터 웨어하우스 설계

Solutions.

- Data Lake
 - 소싱 Tasks에서 저장한 원본 JSON 데이터들이 위치함
 - hive external table에 해당 데이터 경로를 지정하여 처리 엔진에서 접근 가능

```
CREATE TABLE hive.data_{data_id}.order_tmp (  
  orders VARCHAR  
)  
WITH (  
  external_location = 's3a://{bucket_name}/lake/{file_path}/',  
  format = 'TEXTFILE'  
)
```



01. Data Warehouse

데이터 웨어하우스 설계

Solutions.

- Data Warehouse
 - iceberg 테이블이며, parquet 타입으로 저장됨
- {data_name}_src 테이블
 - Data Lake의 table에 스키마를 부여한 테이블
 - Data를 제공하는 플랫폼마다 스키마 구조가 다름

```
CREATE TABLE iceberg.data_{data_id}.order_src
WITH (
  format = 'PARQUET',
  format_version = 2,
  location = 's3a://{bucket_name}/mart/data_{data_id}.db/order_src'
)
AS (
  SELECT JSON_EXTRACT(json_data, '$.orderNo') AS order_no
         , FROM_IS08601_TIMESTAMP(JSON_EXTRACT(json_data, '$.orderDt')) AS order_dt
         -- , other columns ...
  FROM hive.data_{data_id}.order_tmp
)
```

01. Data Warehouse

데이터 웨어하우스 설계

Solutions.

- Data Warehouse
 - iceberg 테이블이며, parquet 타입으로 저장됨
- {data_name} 테이블
 - 공통 스키마를 설계하여 해당 스키마를 따르는 테이블 생성

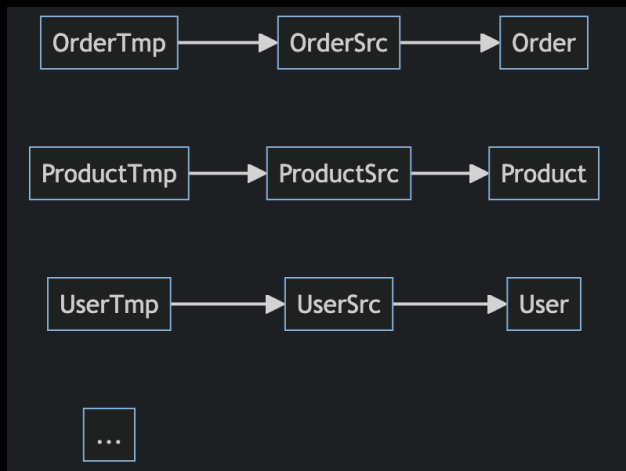
```
CREATE TABLE iceberg.data_{data_id}.order
WITH (
  format = 'PARQUET',
  format_version = 2,
  location = 's3a://{bucket_name}/mart/data_{data_id}.db/order'
)
AS (
  SELECT CAST(order_no AS VARCHAR) AS order_no
        , CAST(order_dt AS TIMESTAMP(6)) AS order_dt
        -- , other columns ...
  FROM hive.data_{data_id}.order_src
)
```


01. Data Warehouse

데이터 웨어하우스 설계

Solutions.

- Data Warehouse
 - 다양한 데이터의 처리 Task를 Airflow로 스케줄

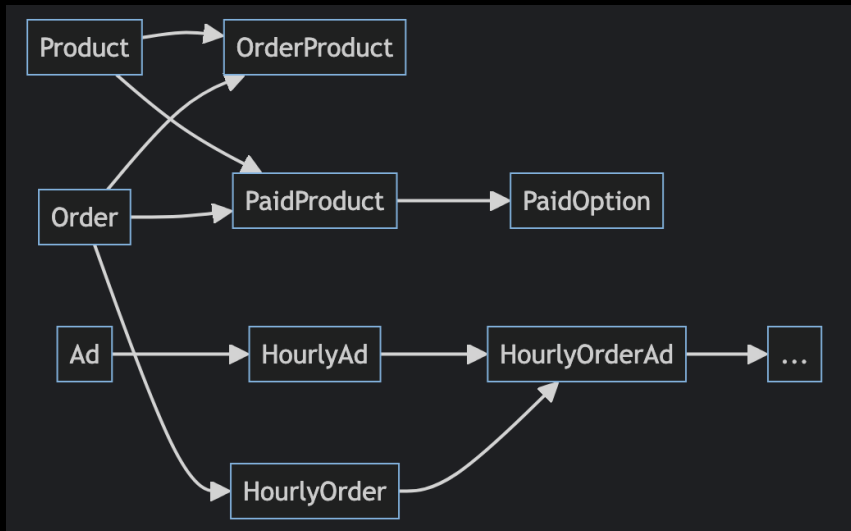


01. Data Warehouse

데이터 웨어하우스 설계

Solutions.

- Data Mart
- Warehouse의 테이블을 이용해 다양한 Mart 생성



01. Data Warehouse

데이터 웨어하우스 설계

Results.

- 중복된 로직 제거 및 테이블 간 데이터 일관성 확보를 통해 스토리지 및 관리 비용 절감, 재사용성 확보
- 테이블의 다운타임 없이 데이터 업데이트를 진행할 수 있게 됨.
- Parquet 데이터의 이점을 활용
 - 특정 컬럼들을 대상으로 쿼리를 요청하는 케이스에서 I/O 비용을 절감하고 쿼리 성능 확보

01. Data Warehouse

item-stat data 모델링

item-stat data 모델링

Problems.

- 아래 스펙의 분석 기능을 대시보드에 추가해야 함

기초 상품 주문	
Dimensions Filters	결제 날짜 시간 + 판매처 관련 + 배송 관련 + 회원 관련 + 상품관련 + 기초상품 관련 + ...
Measures	매출 관련, 광고비 관련

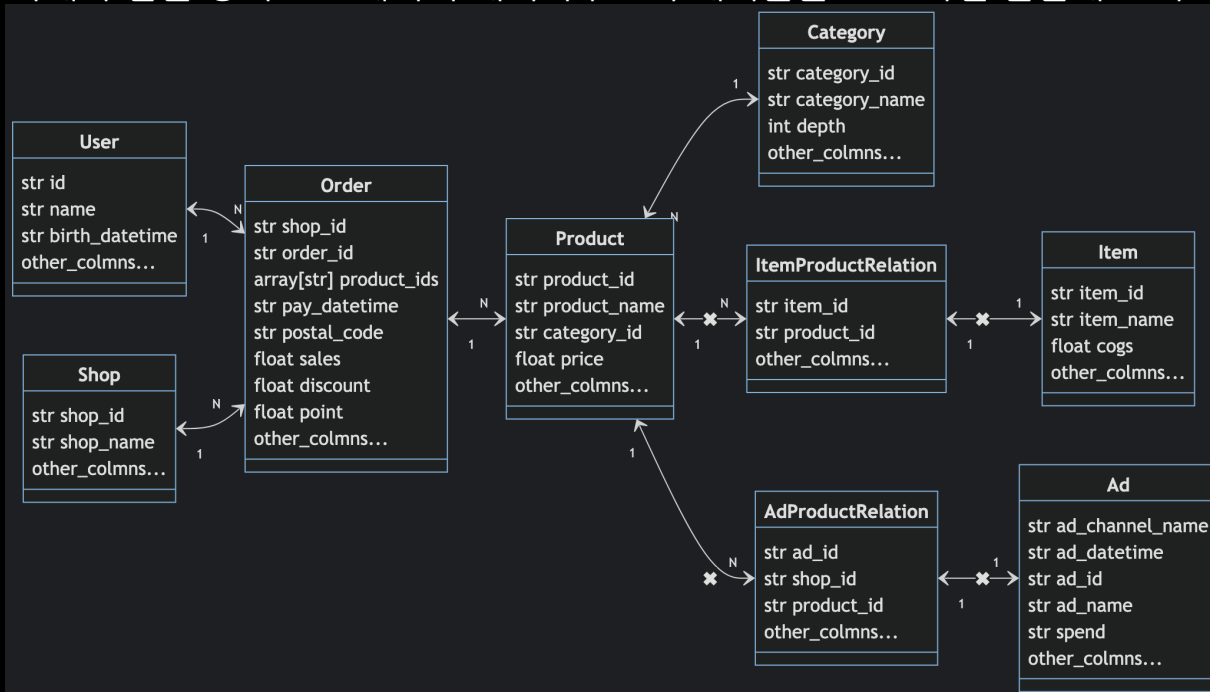
- 예를 들면, A 판매처의 B 기초상품을 쿠팡에서 구매한 경기도 고객의 연령 별 광고비 대비 매출 비율을 구할 수 있어야 함.
- 금액 관련 데이터는 Transform으로 인한 오차가 절대 발생하면 안됨
- 클라우드 환경을 사용하는 만큼 데이터 처리로 인한 서버 비용을 최소화 해야 함
- 위의 요구사항을 모두 만족하는 테이블을 데이터 마트에 추가하기로 결정

01. Data Warehouse

item-stat data 모델링

Problems.

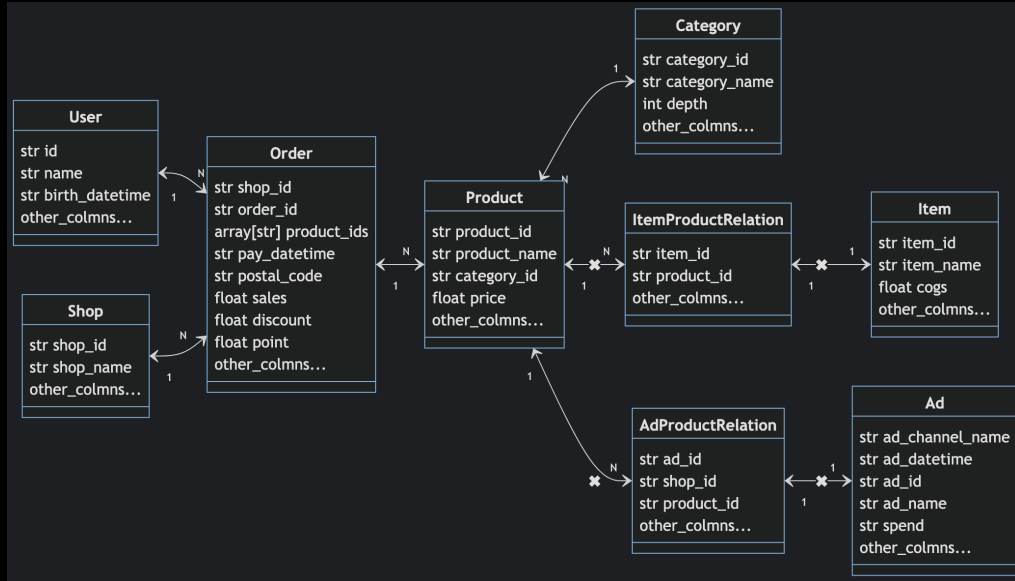
- 아래와 같은 방식으로 데이터 웨어하우스의 테이블을 JOIN하면 간단해 보이지만 문제가 존재함



01. Data Warehouse

item-stat data 모델링

Problems.



- 1: N JOIN 시 sales, discount 등 금액 관련 measure들이 복제되어 매출 데이터의 정합성이 깨짐
 - Order <-> Product, Product <-> Item, Product <-> Ad 에서 발생
- Relation이라는 이름이 붙어있는 관계테이블에는 누락된 매칭 정보가 존재함
 - 서로 다른 플랫폼 데이터의 관계 정보이므로 완벽하기 어려움
- Heavy한 테이블끼리 Join하게 되면서 데이터 량이 방대해짐

01. Data Warehouse

item-stat data 모델링

Solution 1. 1:N JOIN 시 금액 관련 measure를 적절히 분배

1. N에 해당하는 테이블의 금액 관련 measure를 기준으로 분배율을 산정
2. 분배된 measure의 정수 부분과 유리수 부분을 나누고, 유리수의 합을 하나의 row에 몰아넣음
 - 모든 금액 measure를 정수화
 - 실수 연산 시 나타나는 오차를 커버

```
1 WITH with_distribution AS (  
2     SELECT *  
3         , product.price / (SUM(product.price) OVER (PARTITION BY order_id)) AS distribution_rate  
4         , ROW_NUMBER() OVER (PARTITION BY order.order_id ORDER BY product.price) AS rank_by_price_in_order  
5     FROM order  
6     LEFT OUTER JOIN product  
7         ON order.product_id = product.product_id  
8 ), apply_distribution AS (  
9     SELECT *  
10        , sales * distribution_rate AS distributed_sales  
11    FROM with_distribution  
12 ), apply_tuncation AS (  
13     SELECT *  
14        , TRUNCATE(distributed_sales) AS truncated_sales  
15        , distributed_sales - TRUNCATE(distributed_sales) AS sales_remainder  
16    FROM apply_distribution  
17 ), with_remainder AS (  
18     SELECT *  
19        , SUM(sales_remainder) OVER (PARTITION BY order_id) AS sales_remainder_by_order  
20    FROM apply_tuncation  
21 )  
22 SELECT *  
23        , IF(rank_in_order = 1, truncated_sales + ROUND(sales_remainder_by_order), truncated_sales) AS order_product_sales  
24 FROM with_remainder
```


01. Data Warehouse

item-stat data 모델링

Solution 2. Relation 데이터의 상태에 따라 적절히 매칭 (Order <-> Ad 매칭 예시)

1. 매칭될 경우

1. shop, product 단위로 매칭될 경우
2. shop 단위로만 매칭될 경우

2. 매칭 정보는 있지만 동일한 time 구간에 주문 데이터가 없는 경우
3. 매칭 정보가 없을 경우

- 위의 모든 경우의 수에 해당하는 CTE들을 생성 후 UNION

- Order의 각 row에 광고비 데이터가 매출에 비례해서 적절히 분배됨

```
WITH matched_ad_by_shop AS (  
    SELECT * FROM daily_ad_with_relation  
    WHERE shop_id IS NOT NULL  
    AND product_id IS NULL  
) , matched_ad_by_shop_product AS (  
    SELECT * FROM daily_ad_with_relation  
    WHERE shop_id IS NOT NULL  
    AND product_id IS NOT NULL  
)  
  
...  
  
SELECT * FROM hourly_order_with_shop  
INNER JOIN matched_ad_by_shop  
    ON hourly_order_with_shop.shop_id = matched_ad_by_shop.shop_id  
  
UNION ALL  
  
SELECT * FROM hourly_order_with_shop  
INNER JOIN matched_ad_by_shop_product  
    ON hourly_order_with_shop.shop_id = matched_ad_by_shop.shop_id  
    AND hourly_order_with_shop.product_id = matched_ad_by_shop.product_id  
  
...
```

01. Data Warehouse

item-stat data 모델링

Solution 3. Roll up, 파티셔닝을 사용

1. 요구사항에 따른 최소 time frequency로 롤업

- hourly(time_id)와 dimensions를 기준으로 모든 measures를 롤업
- Join 및 추가 집계 처리량을 최적화

```
CREATE TABLE hourly_item_order AS (  
    SELECT DATE_TRUNC('HOUR', pay_datetime) AS time_id  
        , product_name  
        , option_name  
        , item_name  
        -- , other dimensions ...  
        , SUM(sales) AS sales  
        , SUM(delivery_fee) AS delivery_fee  
        -- , other measures ...  
    FROM item_order  
    GROUP BY 1, 2, 3, 4, ...  
)
```

01. Data Warehouse

item-stat data 모델링

Solution 3. Roll up, 파티셔닝을 사용

2. MONTH(time_id)를 기준으로 파티셔닝

- 데이터가 주로 시간 기반으로 조회됨.
- 데이터가 시간을 기준으로 균등하게 분포 되어있음

```
CREATE TABLE hourly_item_order
WITH (
    FORMAT = 'PARQUET',
    FORMAT_VERSION = 2,
    LOCATION = 's3a://{bucket_name}/mart/data_{data_id}.db/hourly_item_order',
    PARTITIONING = ARRAY['MONTH(time_id)']
)
AS (
    SELECT DATE_TRUNC('HOUR', pay_datetime) AS time_id
           , product_name
           , option_name
           , item_name
           -- , other dimensions ...
           , SUM(sales) AS sales
           , SUM(delivery_fee) AS delivery_fee
           -- , other measures ...
    FROM item_order
    GROUP BY 1, 2, 3, 4, ...
)
```

01. Data Warehouse

item-stat data 모델링

Solution 3. Roll up, 파티셔닝을 사용

3. 롤업 시 깨지는 데이터가 존재

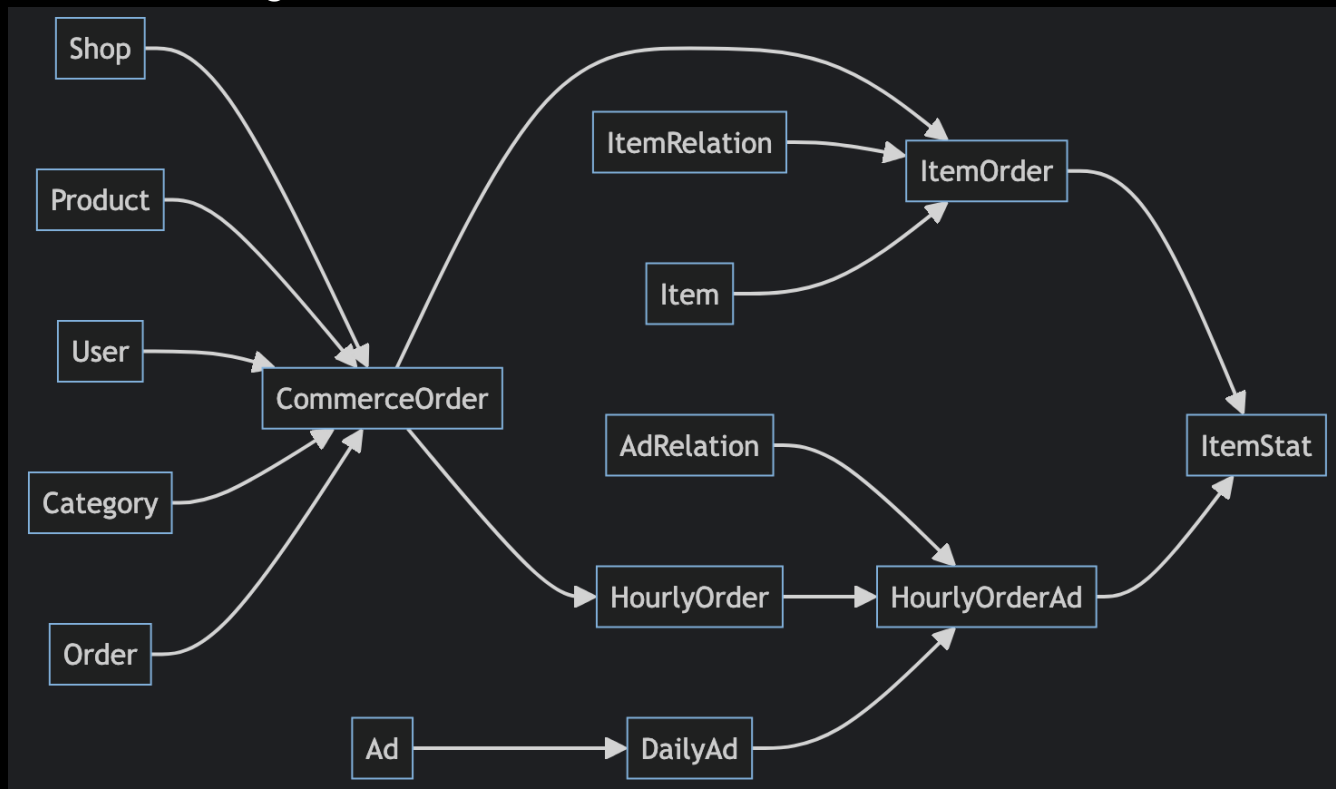
- 예시로, order_id 데이터는 카디널리티가 너무 높음
→ 롤업 대상에서 제외
- COUNT(DISTINCT order_id) 같은 연산을 수행할 수 없음
- 대시보드 서버에서 필요한 요청을 적용한 가벼워진 데이터를 대상으로 연산을 런타임에 수행한 후
롤업 테이블과 JOIN 되도록 설계하여 해결
- 롤업 집계 시 HyperLogLog 등의 추산 집계를 하는 방법도 있지만,
금액 데이터는 정확성이 절대 깨지면 안되므로 사용하지 않음

```
-- REQUEST
-- frequency -> 일별 분석
-- measures -> 결제 건수, 매출
-- dimensions -> product_name, option_name
-- filters -> product_name = 'A'

WITH from_item_order AS (
    SELECT DATE_TRUNC('DAY', pay_datetime) AS time_id
           , product_name
           , option_name
           , COUNT(DISTINCT order_id) AS pay_cnt
    FROM item_order
    WHERE product_name = 'A'
    GROUP BY DATE_TRUNC('DAY', pay_datetime)
           , product_name
           , option_name
), from_hourly_item_order AS (
    SELECT DATE_TRUNC('DAY', time_id) AS time_id
           , product_name
           , option_name
           , SUM(sales) AS sales
    FROM hourly_item_order
    WHERE product_name = 'A'
    GROUP BY DATE_TRUNC('DAY', pay_datetime)
           , product_name
           , option_name
)
SELECT from_hourly_item_order.time_id
      , from_hourly_item_order.product_name
      , from_hourly_item_order.option_name
      , from_hourly_item_order.sales
      , from_item_order.pay_cnt
FROM from_hourly_item_order
LEFT OUTER JOIN from_item_order
ON (
    from_hourly_item_order.time_id = from_item_order.time_id
    AND from_hourly_item_order.product_name = from_item_order.product_name
    AND from_hourly_item_order.option_name = from_item_order.option_name
)
```

item-stat data 모델링

Result. 최종 Dag



01. Data Warehouse

item-stat data 모델링

Result.

- 데이터 처리 엔진의 성능 및 스펙 타협 없이 요구사항을 모두 만족할 수 있었음.
- 데이터 정합성 확보
 - Order -> product -> option -> item 순서로 쪼개진 데이터 임에도 불구하고
 - Order data에서 매출을 집계했을 때와 비교해도 정확히 같고
 - Ad data에서 광고비를 집계했을 때와 비교해도 정확히 같음

02. Data Pipeline

Data Compaction

02. Data Pipeline

Data Compaction

Problems.

1. 수집한 커머스 데이터를 data lake에 저장 및 hive external table 생성
2. data lake의 데이터에 스키마를 부여하고 data warehouse에 iceberg 테이블로 저장
 - 해당 작업의 속도가 너무 느려서 bottleneck이 됨.

```
CREATE TABLE hive.data_{data_id}.order_tmp (  
    orders VARCHAR  
)  
WITH (  
    external_location = 's3a://{bucket_name}/lake/{file_path}/',  
    format = 'TEXTFILE'  
)
```

```
CREATE TABLE iceberg.data_{data_id}.order_src  
WITH (  
    format = 'PARQUET',  
    format_version = 2,  
    location = 's3a://{bucket_name}/mart/data_{data_id}.db/order_src'  
)  
AS (  
    WITH parsed_order AS ( -- Parse TEXT to JSON  
        SELECT CAST(JSON_EXTRACT(orders, '$.data') AS array(JSON)) AS orders  
        FROM hive.data_{data_id}.order_tmp  
    ), exploded_order AS ( -- Explode list to rows  
        SELECT json_data  
        FROM parsed_order  
        CROSS JOIN UNNEST(orders) AS t(json_data)  
    )  
    SELECT CAST(JSON_EXTRACT(json_data, '$.orderNo') AS VARCHAR) AS order_no  
        , FROM_ISO8601_TIMESTAMP(JSON_EXTRACT(json_data, '$.orderDt')) AS order_dt  
        -- other columns...  
    FROM exploded_order  
)
```


02. Data Pipeline

Data Compaction

Solutions.

- 원인 파악

- data lake의 한 데이터는 약 10GB 정도이며, 200KB ~ 4MB의 JSON 파일들로 구성

```
{
  "data": [
    {
      "orderNo": "20241115-0009380",
      "orderDt": "2024-11-15T08:48:49",
      ~~~~~
    },
    ~~~~~
  ]
}
```

- I/O + JSON 파싱 비용

- 모든 JSON 파일의 'data' 컬럼을 대상으로 I/O와 JSON파싱 작업을 수행 해야함

- Explode 비용

- List 내에 나열되어 있는 JSON을 Table로 만들기 위해 Explode를 수행 해야 함

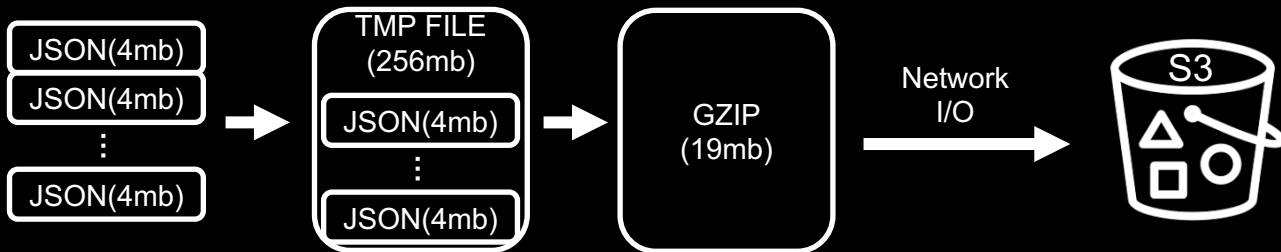
- Explode 수행 시 내부적으로 크로스 조인이 발생하여 메모리 사용량과 처리 시간이 크게 증가

02. Data Pipeline

Data Compaction

Solutions.

- 해결 방안
 - Data 소싱 시 약 256MB 만큼의 데이터를 버퍼에 쌓아서 Gzip 압축 후 Bulk로 저장
 - 파일의 개수와 크기를 전부 대폭 줄여서 I/O 오버헤드 절약
 - temporary file을 데이터를 쌓는 버퍼로 사용
 - 병렬 작업에서 메모리 부담을 줄임
 - 데이터를 line-delimited JSON으로 저장
 - 데이터 웨어하우스에 저장할 때 Explode 작업을 수행해도 되지 않아서 처리 효율 대폭증가



02. Data Pipeline

Data Compaction

Results.

	총 파일 수	총 용량	테이블 생성 시간
AS - IS	3650	10,388 MB	7분 30초
TO - BE	50	772 MB	40초
RESULT	약 98% 감소	약 92.5% 감소	약 91% 감소

파일의 개수와 크기, 쿼리엔진의 처리량을 모두 최적화 하여 테이블 생성 시의 병목 현상을 해결할 수 있었음

02. Data Pipeline

Spark - Dynamic Resource Allocation 도입

Spark - Dynamic Resource Allocation 도입

Problems.

데이터 처리 로직을 Spark로 운용하면서 몇 가지 문제 존재

- 커머스 데이터 특성 상 마케팅이나, 할인 행사 등으로 인해 데이터 양이 급증하는 경우가 있음.
 - 작업 시간 지연
 - OOME 발생
- 반대로, 데이터 양이 너무 적은 경우도 있음.
 - 처리량에 비해 과도한 리소스 비용 지출
 - 작업이 일찍 끝났음에도 종료되지 않는 유휴 Executor 존재

02. Data Pipeline

Spark - Dynamic Resource Allocation 도입

Solutions.

- DRA 활성화
 - 작업 부하에 맞춰 Executor를 동적으로 조절
- Executor의 확장 및 축소 범위 조절
 - Airflow의 Variable과 UDM을 이용하여 데이터의 양에 비례해서 Executor의 initial, min, max값을 동적으로 설정
- Timeout 및 Garbage Collection 설정
 - Executor의 Idle 상태에 대한 Timeout을 설정하여 유휴 Executor를 빠르게 회수

```
EmrContainerOperator(  
    task_id=TMP_TO_SRC_ORDER_TASK_ID,  
    # other settings ...,  
    configuration_overrides={  
        "spark.dynamicAllocation.enabled": "true",  
        "spark.dynamicAllocation.shuffleTracking.enabled": "true",  
        "spark.dynamicAllocation.minExecutors": "{{ udm_predict_executors(data_cnt, 'min') }}",  
        "spark.dynamicAllocation.maxExecutors": "{{ udm_predict_executors(data_cnt, 'max') }}",  
        "spark.dynamicAllocation.initialExecutors": "{{ udm_predict_executors(data_cnt, 'initial') }}",  
        "spark.dynamicAllocation.executorIdleTimeout": "60s",  
        # other settings ...,  
    },  
    # other settings ...,  
)
```

02. Data Pipeline

Spark - Dynamic Resource Allocation 도입

Results.

- 리소스 효율성 향상
 - 작업량이 많은 경우에는 최대 32개의 Executor까지 확장되었고,
작업이 완료되거나 유휴 상태에 접어들면 최소 2개의 Executor로 축소
- 비용 절감 & 성능 최적화 모두 달성
 - 리소스 부족으로 인해 task가 failure 하거나, 지연되는 문제 해소
 - EMR을 운용하는데 드는 월 비용 약 20% 감소

03. Dashboard Serve

대시보드 API 로딩 속도 개선 (계층 별 캐싱)

대시보드 API 로딩 속도 개선 (계층 별 캐싱)

Problems.

- Heavy한 집계 요청을 수행할 경우 30초 이상의 시간이 소요되어 Timeout 발생
- 로그 통계를 보았을 때 비슷한 요청의 빈도가 잦음
 - API 서버와 Trino에 많은 부하를 줌
- 단순히 Trino의 스펙을 높여서 성능을 개선할 수 있었지만, 이는 클라우드 컴퓨팅 자원의 비용 상승으로 이어짐

대시보드 API 로딩 속도 개선 (계층 별 캐싱)

Solutions 1. API 서버 캐싱

- 서버로 들어오는 Request를 key로 잡아, Response를 캐싱하는 데코레이터 개발
- Redis를 캐시 서버로 활용

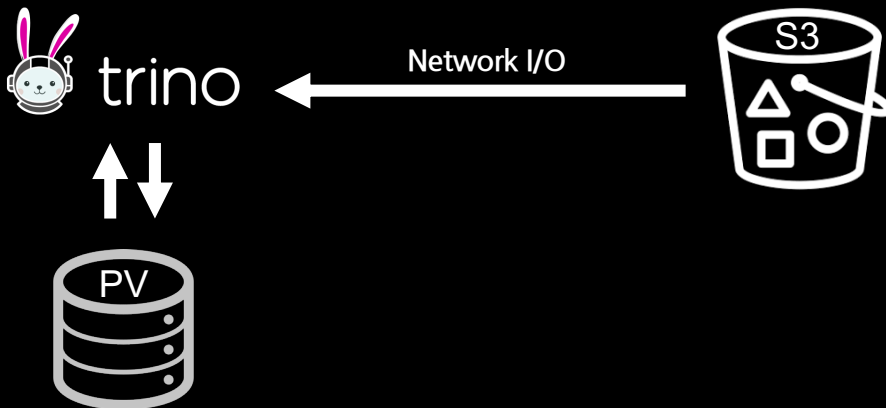
캐시 키	"{user_id}:{method}:{url}:{hashlib.sha256(body).hexdigest()}"
캐시 값	Pickle 직렬화 후 gzip 압축된 Response
메모리 초과 시	TTL이 설정된 키 중에서 LRU 기준으로 eviction
TTL	50분

- 유저의 모든 캐시를 eviction하는 로직을 hourly data update event에 대한 핸들러로 등록

대시보드 API 로딩 속도 개선 (계층 별 캐싱)

Solutions 2. Trino 캐시

- 생성된 대시보드의 통계 확인 결과,
동일한 데이터 소스에 다양한 집계를 설정한 다수의 차트를 하나의 대시보드에 위치 시키는 경향이 있음
 - 동일한 파일을 중복해서 read하는 일이 잦다는 것을 의미함
- Trino가 데이터 웨어하우스와 레이크에서 읽어온 데이터를 노드의 PV에 캐시 하도록 설정



대시보드 API 로딩 속도 개선 (계층 별 캐싱)

Result.

- API 서버 단위 캐싱으로 인해 API 서버에서 Trino에 전달되는 쿼리량이 60% 이상 감소
- Trino 캐시로 인해 Trino와 S3 사이의 트래픽 비용 및 처리 시간을 대폭 절감
- 대시보드 API의 평균 응답 속도를 5초 이내로 단축
- 인프라 비용의 증가 없이 성능 최적화를 달성할 수 있었음

감사합니다.